

Langchain core components

https://dev.to/shreya_ghorui/langchain-from-scratch-a-complete-beginners-guide-with-diagrams-4aoo

LangChain is a tool for builders.

🔗 **Quick Analogy:** Think of Foundation Models like a powerful industrial oven. A *user* just bakes bread in it. A *builder* designs the entire bakery — the recipes, the assembly line, the packaging — using that oven as the core engine. LangChain is your bakery blueprint.

Why LangChain First?

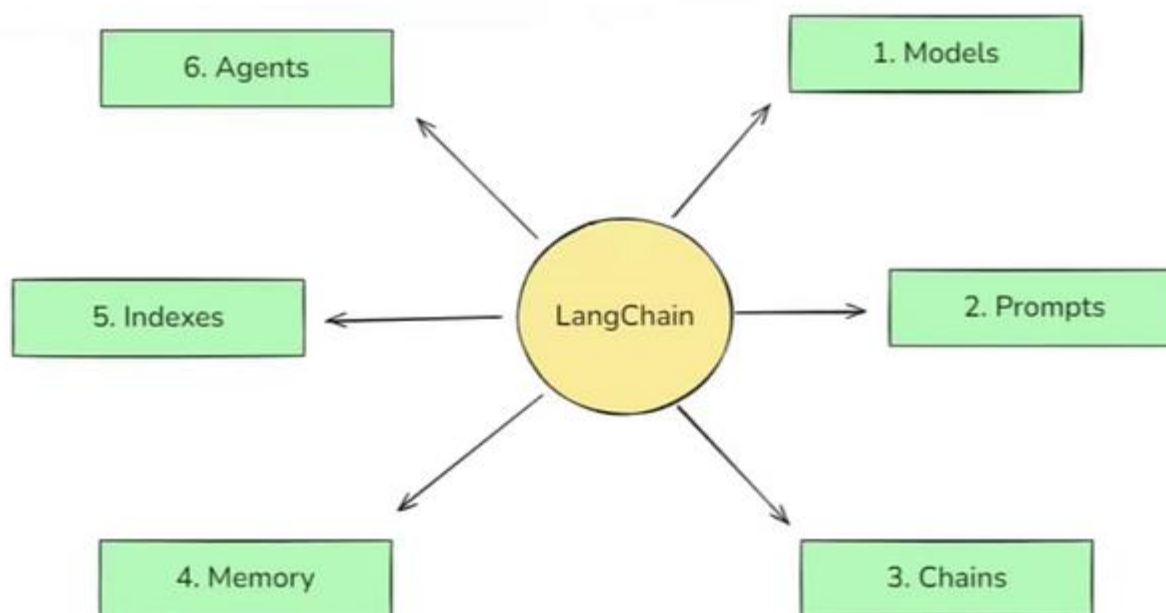
LangChain is an open-source framework that helps in building LLM-based applications. It provides modular components and end-to-end tools that help developers build complex AI applications such as chatbots, question-answering systems, retrieval-augmented generation (RAG), autonomous agents, and more.

Key Benefits

1. Supports all the major LLMs
 2. Simplifies developing LLM-based applications
 3. Integrations available for all major tools
 4. Open source / Free / Actively developed
 5. Supports all major GenAI use cases
- 💡 One framework. Every model. Every use case. That's the LangChain promise.

LangChain Components

LangChain is built around **6 core components** — think of them as the organs of a body, each with a specific job, all working together:



0. Quick Reference — The 7 Core Concepts

Before the deep dive, the simplest possible map of what LangChain is built from:

Concept	What it does	When you need it
Chain (LCEL)	Composes LLM calls with data transformations using <code> </code>	Every LLM application
Prompt Template	Structures input sent to the LLM	Whenever you need consistent, reusable prompts
Retriever	Fetches relevant documents from a data source	RAG applications
Agent	LLM decides which tools to call, and in what order	Dynamic, multi-step workflows
Tool	A function the agent is allowed to call	When the agent needs to <i>do</i> something external
Memory	Stores conversation history	Chatbots, multi-turn interactions
Output Parser	Converts free LLM text into typed/structured data	Whenever downstream code needs a specific shape

Rule of thumb: every LLM app needs Chain + Prompt Template. Add a Retriever for RAG. Add Tool + Agent once the app needs to take actions, not just answer.

LangChain Rule of Thumb — Layered Complexity

Layer	Add This	What It Does	When You Need It	Why
1. Baseline	Chain (LCEL) + Prompt Template	Structures input for the model and wires "input → LLM → output" into a single flow	Every LLM application, no exceptions	This is the minimum viable pipeline — nothing simpler exists
2. Knowledge-Grounded	+ Retriever	Fetches relevant data from your own documents/database so the model answers using real context instead of guessing	RAG apps — e.g. "chat with your PDF" or internal knowledge base	Prevents hallucination when answers must reflect your specific data, not just training knowledge
3. Action-Taking	+ Tool + Agent	Tool = a function the model can call (API, database, calendar). Agent = decides which tool to use and when, dynamically	Apps that must <i>do</i> something — send emails, query live systems, take multi-step actions	A static Chain runs the same fixed sequence every time; an Agent adapts based on the situation

Rule of Thumb (summary)

Every app needs Chain + Prompt Template → add Retriever for RAG → add Tool + Agent for actions, not just answers.

1. Models

→ **What it is:** The interface to talk to any AI model.

In LangChain, *models* are the interfaces through which you interact with AI models.

The evolution of language models: NLP → NLU → LLMs → Internet scale (Billions of params, >100GB)

The Problem without LangChain:

Every provider (OpenAI, Anthropic, HuggingFace) has its own SDK, its own syntax, its own quirks. Switching models means rewriting your entire codebase.

The LangChain Solution:

A single, unified `model.invoke()` interface — regardless of the provider.

```
# OpenAI via LangChain
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()
model = ChatOpenAI(model='gpt-4', temperature=0)
result = model.invoke("How divide the result by 1.5?")
print(result.content)
```

```
# Anthropic Claude via LangChain — same interface, different model!
from langchain_anthropic import ChatAnthropic
from dotenv import load_dotenv

load_dotenv()
model = ChatAnthropic(model='claude-3-opus-20240229')
result = model.invoke("Hi who are you")
print(result.content)
```

💡 **Key idea:** Swap `ChatOpenAI` for `ChatAnthropic` and everything else stays the same. That's model-agnostic development.

✦ **What Happens Next?** Once you have a model, you need to talk to it intelligently. That's where **Prompts** come in.

2. 🗨️ Prompts

→ **What it is:** Reusable, dynamic templates for talking to LLMs.

LLMs take **input** → **prompt** → **output**. A raw string works, but it's fragile. LangChain makes prompt management powerful, reusable, and structured.

🔗 **Quick Analogy:** A raw string prompt is like shouting an order at a chef. A `PromptTemplate` is like handing them a proper recipe card — structured, consistent, and repeatable every time.

→ Step 1: *Dynamic & Reusable Prompts*

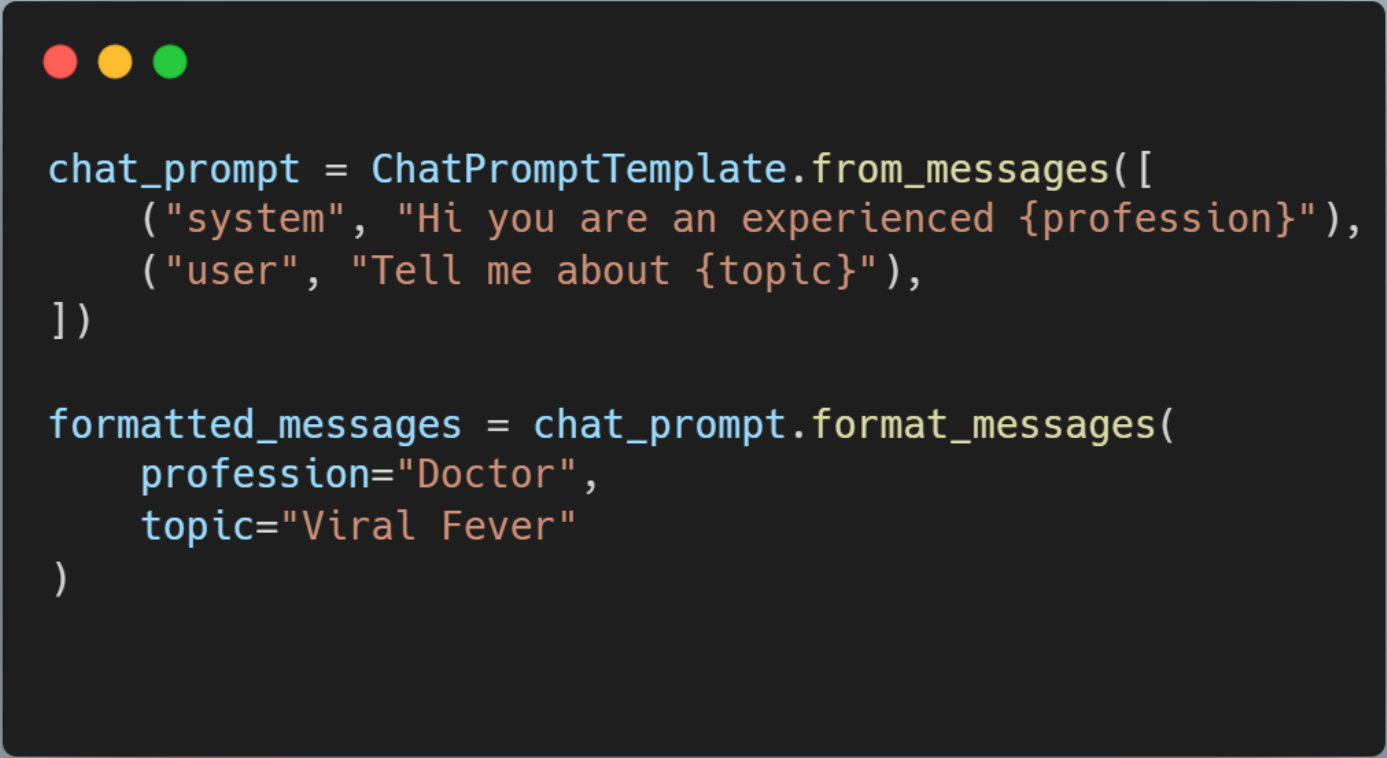
```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template(
    "Summarize {topic} in {emotion} tone"
)

print(prompt.format(topic="Cricket", emotion="fun"))
```

Step 2: Role-Based Prompts

Give your LLM a persona — like a Doctor, a Lawyer, or a Code Reviewer:



```

chat_prompt = ChatPromptTemplate.from_messages([
    ("system", "Hi you are an experienced {profession}"),
    ("user", "Tell me about {topic}"),
])

formatted_messages = chat_prompt.format_messages(
    profession="Doctor",
    topic="Viral Fever"
)

```

Method	Use Case	Input
from_template()	Single-turn, plain-text prompt	One string with {variables}
from_messages()	Multi-turn chat prompt (system/user/assistant roles)	List of (role, text) tuples

→ *Step 3: Few Shot Prompting*

Teach the model by example — show it what "good output" looks like before asking your real question:

```

examples = [
    {"input": "I was charged twice for my subscription this month.", "output": "Billing Issue"},
    {"input": "The app crashes every time I try to log in.", "output": "Technical Problem"},
    {"input": "Can you explain how to upgrade my plan?", "output": "General Inquiry"},
    {"input": "I need a refund for a payment I didn't authorize.", "output": "Billing Issue"},
]

example_template = """
Ticket: {input}
Category: {output}
"""

few_shot_prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=PromptTemplate(
        input_variables=["input", "output"],
        template=example_template
    ),
    prefix="Classify the following customer support tickets into one of the categories: "
        "'Billing Issue', 'Technical Problem', or 'General Inquiry'.\n\n",
    suffix="Ticket: {user_input}\nCategory:",
    input_variables=["user_input"],
)

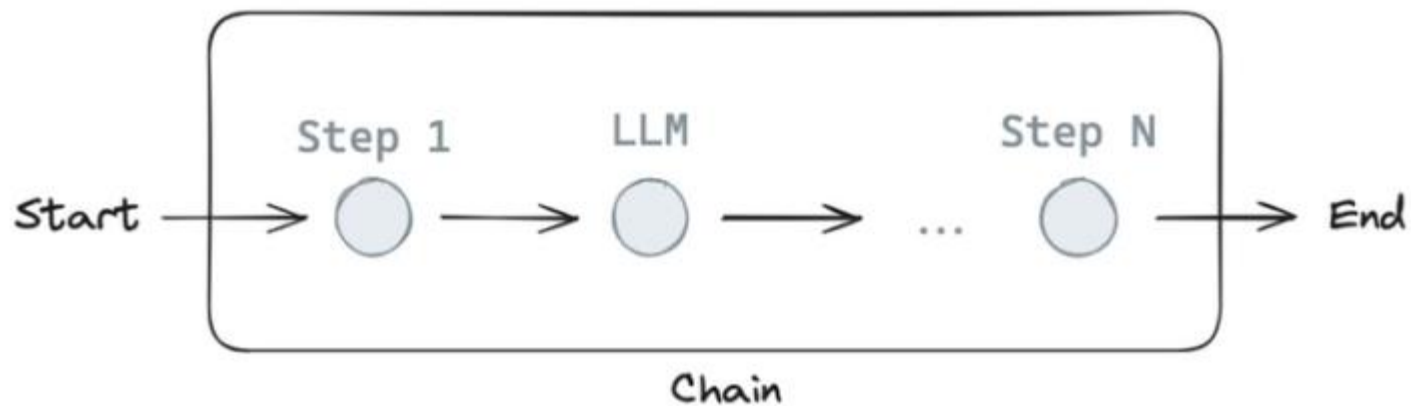
# To actually use it:
final_prompt = few_shot_prompt.format(user_input="My payment page keeps freezing.")
print(final_prompt)

```

3. ∞ Chains

→ **What it is:** Pipelines that connect LLMs with other components.

Chains = Pipelines. They are the heart of LangChain (hence the name!). Instead of calling a model once, you chain multiple calls and operations together into a workflow.

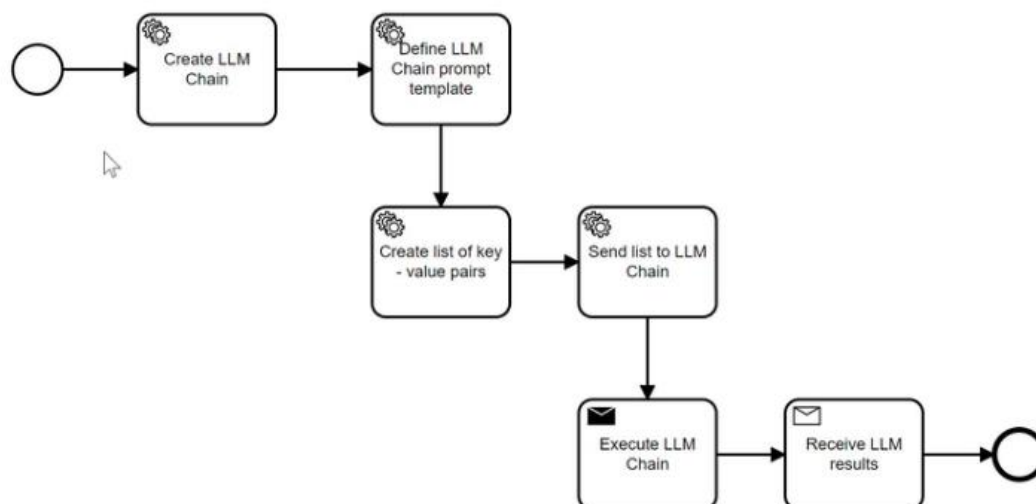


🔗 **Quick Analogy:** A single LLM call is like one chef making one dish. A Chain is the entire restaurant kitchen — prep cook → head chef → plating station — each step feeding the next.

Types of Chains

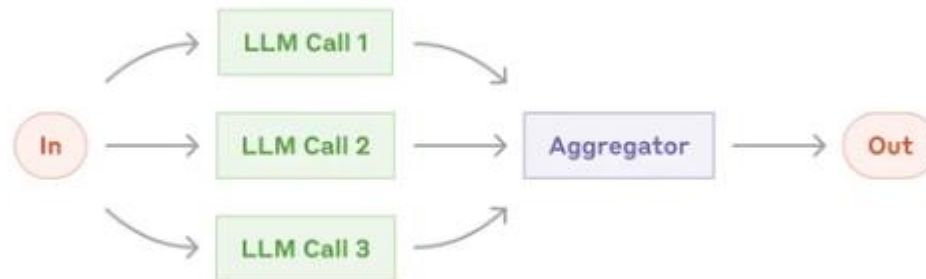
1. **Sequential Chains** — Steps run one after another:

Example: Translate 1000-word English text → Hindi summary (100 words)



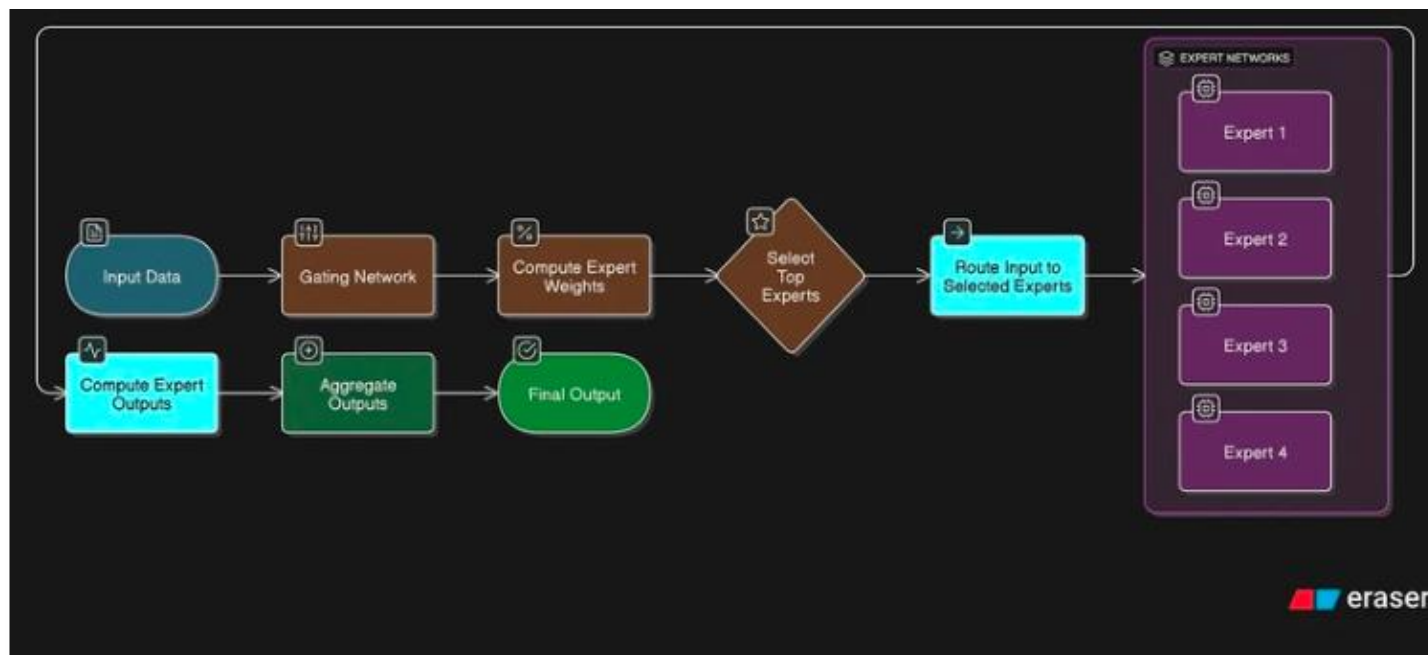
2.Parallel Chains — Multiple LLMs run simultaneously, results combined:

Example: Generate a report from two expert LLMs simultaneously, then merge



3 .Conditional Chains — Route based on output:

Example: AI feedback agent — good feedback → "Thank you!", bad feedback → send email alert



✦ **What Happens Next?** Chains are stateless — they don't remember previous conversations. To build a real chatbot, we need **Memory**.

4. 🧠 Memory

→ **What it is:** Giving your LangChain app the ability to remember.

Without memory, every API call is stateless — like talking to someone with amnesia who forgets you the moment you stop speaking.

LangChain's memory components let you **persist and retrieve conversation history**, making chatbots feel natural and context-aware.

🔍 **Quick Analogy:** Memory is like a notepad your assistant keeps on the desk. Every time you talk, they jot down what was said. Next time you walk in, they already know your name, your preferences, and what you discussed last week.

✦ **What Happens Next?** Memory handles conversation history — but what if your app needs to search through *thousands of your own documents*? That's where **Indexes (RAG)** come in.

5. 📖 Indexes — The Power of RAG

→ **What it is:** Connecting your LLM to external knowledge.

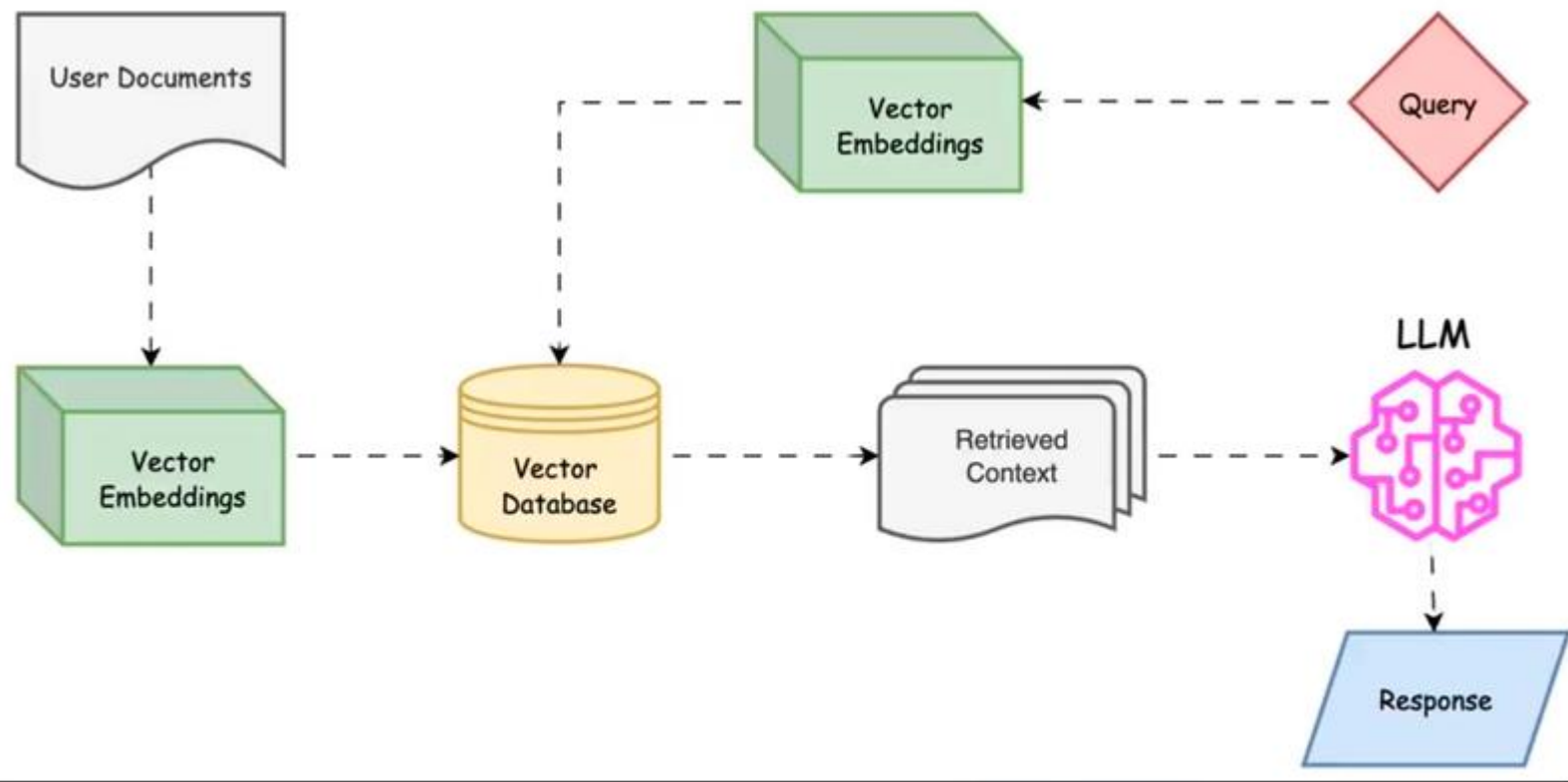
"Indexes connect your application to external knowledge — such as PDFs, websites, or databases."

This is the foundation of **RAG (Retrieval Augmented Generation)** — the most powerful pattern in modern AI apps.

The Problem: LLMs are trained on general internet data. They know nothing about *your* company's internal documents, *your* codebase, or *your* PDF notes.

The RAG Solution: Don't fine-tune the model — just give it your documents at query time.

Step 1: The Full RAG Pipeline

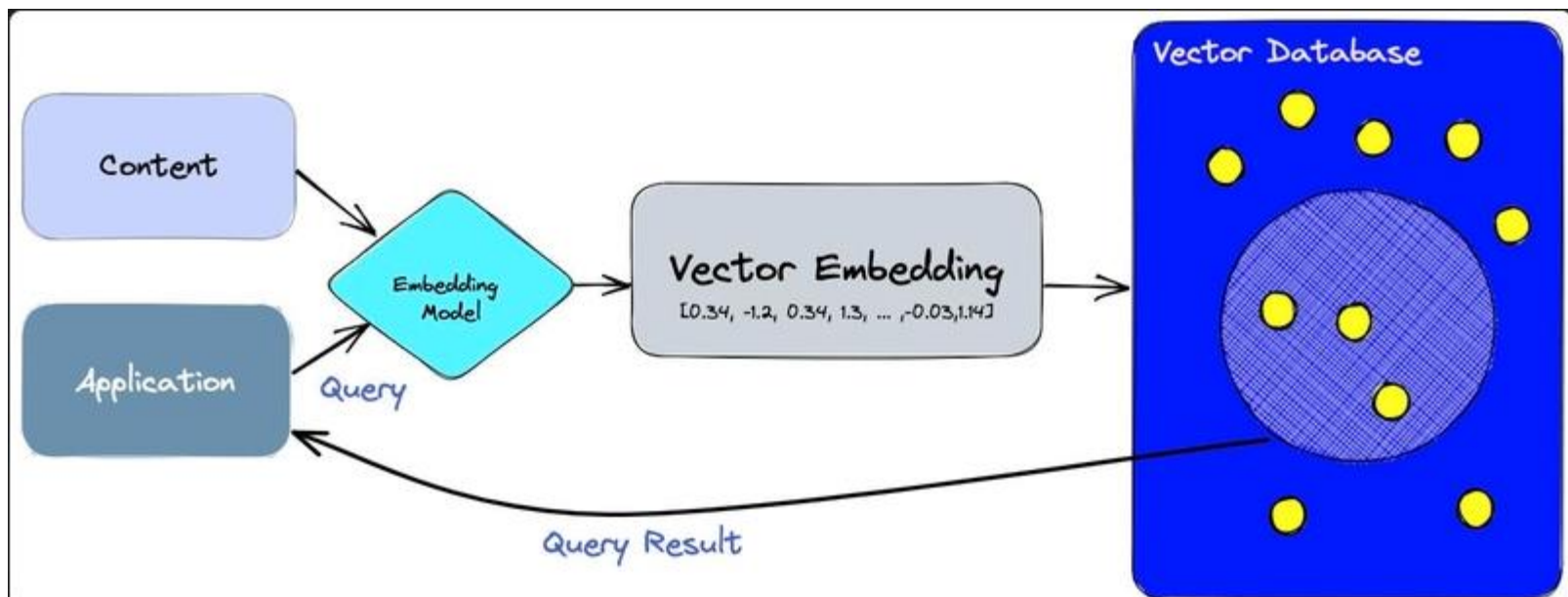


Here's what's happening step by step:

→ *Step 2: Understanding Embeddings & Semantic Search*

Traditional search: keyword matching → "Virat" → [372, 961] (just index positions)

Semantic search converts text into **vectors** — high-dimensional numbers that capture *meaning*, not just spelling.



💡 **Key insight:** "How many runs?" and "total score of" mean the same thing — semantic search finds both. Keyword search finds neither unless the exact word matches.

Search type	How it matches	"How many runs?" vs "total score of"
Keyword search	Matches exact words/tokens (e.g. BM25, Elasticsearch)	Finds neither — no shared words: "runs," "score," "total" don't overlap
Semantic search	Matches <i>meaning</i> , via vector embeddings	Finds both — both queries live near the same region in embedding space (they're asking about the same underlying concept, even worded differently)

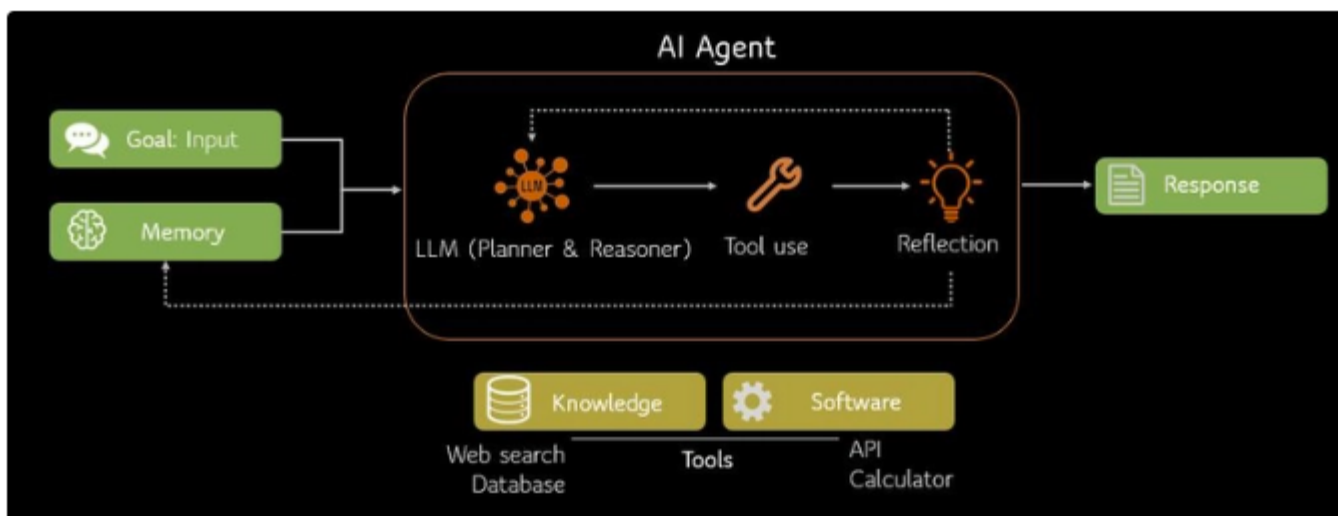
✦ **What Happens Next?** RAG gives your app access to documents. But what if you want your app to *act* — search the web, call an API, book a flight? That's what **Agents** do.

6. 🤖 Agents

➡ **What it is:** LLMs that can think, plan, and use tools.

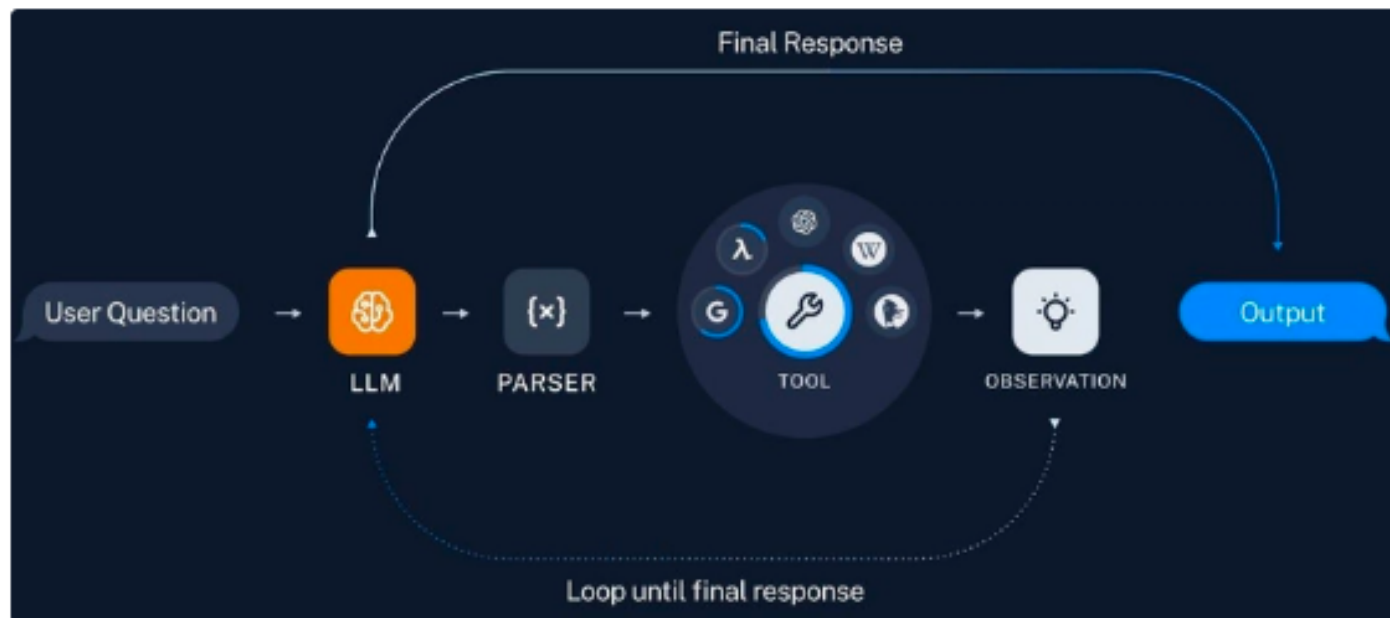
Agents are **AI systems** that combine:

- 🧠 **Reasoning capabilities** (the LLM brain — chain of thought)
- 🔧 **Tools** (external actions it can call)



🔍 **Quick Analogy:** A chatbot is like a very knowledgeable librarian — they can answer questions from memory. An AI Agent is like a personal assistant with a phone — they can answer questions *and* actually call the airline, book the hotel, and send you a confirmation.

How Agents Work



→ Step-by-Step Example

"Can you multiply today's temperature in Delhi with 3?"

Step 1: Agent reasons — "I need Delhi's current temperature. I have a Weather API tool."

Step 2: Agent calls **Weather API** → gets Delhi temp: **32°C**

Step 3: Agent reasons — "Now I need to multiply 32×3 . I have a Calculator tool."

Step 4: Agent calls **Calculator** → **96**

Step 5: Agent returns: "Today's temperature in Delhi is 32°C. Multiplied by 3 = **96."



No hardcoding. No manual steps. Pure autonomous reasoning.

🚀 **What Happens Next?** Now that you know all 6 components, let's see what you can actually *build* with them!

What Can You Build with LangChain?

Application Type	Real Example
Conversational Chatbots	Scalable customer support bot that handles 10,000 queries/day
AI Knowledge Assistants	Q&A over your company's 500-page internal docs
AI Agents	"Make my trip" — searches flights, books hotels, sends itinerary
Workflow Automation	Multi-step pipelines: scrape → summarize → email → log
Summarization/Research Helpers	ChatPDF, research paper summarizer, legal doc analyzer

Prompt Engineering

Simple Notes with Euri API Code Examples

https://github.com/raffeemdai/LangChain_LangGraph/blob/main/prompt_engineering_notes_euri.md

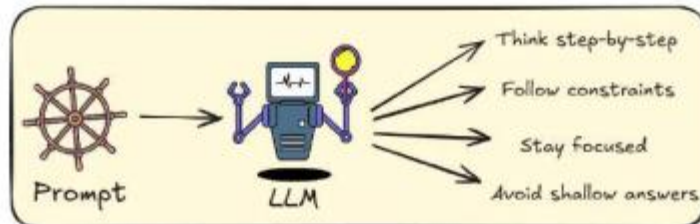
<https://directai.blog/2025/03/26/gen-ai-developer-classroom-notes-26-mar-2025/>

https://github.com/GenAIDevelopment/agenticaiblob/main/june26/rag/rag_learning_gcp/prompts_models_structured_output.ipynb

<https://www.dailydoseofds.com/>

What is Prompt Engineering?

LLMs are powerful, but they don't automatically know what you want. Prompt engineering is the simplest way to control them.



Think of it as the steering wheel for the LLM.

Small adjustments completely shift the direction of the output.

You're not changing weights (the learned parameters inside the model). You're changing instructions and that changes everything.

A good prompt helps the model:

- Think step-by-step
- Follow constraints
- Stay focused
- Avoid shallow answers

It's the fastest, lowest-effort way to get better results from any model.

Let's explore three prompting techniques that significantly improve an LLM's reasoning ability.

Role of Prompt Engineering

- A prompt is a text that contains a set of instructions or requests sent to an AI model, usually referred to as a Large Language Model (LLM).
- Prompt Engineering is the process of designing and refining prompts that guide the behavior of an AI model to produce the desired outputs or responses.

Basic Prompting Rules

Here are 5 basic prompting rules to write effective prompts:

- Assign a Persona
- Core task and subtasks
- Provide Context

- Explain expected output
- Describe any potential limitations

Example:

You are an expert personal finance advisor. Explain different types of investment options available in India. Assume the audience has basic banking schemes knowledge. Provide the explanation and give me the tabular output of all options. Focus on the data from last 5 years.

Prompting Techniques

1. Zero-Shot Prompting

Definition: In zero-shot prompting, you ask the model to complete a task or answer a question without providing any examples or prior context. The model relies solely on its training to understand and generate a response.

Asking a language model to perform a task using only an instruction, with no examples of the task provided in the prompt. The model draws entirely on patterns learned during training to infer what's being asked and how to respond.

Example:

Prompt: "What is the capital of Japan?"

Explanation: This is a zero-shot prompt because you didn't provide any examples or additional context. You simply asked the model a question, and it is expected to know the answer (which is Tokyo).

Expected Output: "Tokyo"

2. Few-Shot Prompting

Definition: In few-shot prompting, you give the model a few examples to show how to perform a task before asking it to do something similar. These examples guide the model and help it understand the pattern or format of the task.

Providing the model with a small number of input-output example pairs (typically 2–5) directly in the prompt before giving it the actual task. This is a form of "in-context learning" — the model isn't retrained, but it uses the examples to infer the pattern, format, style, or reasoning it should follow.

Example:

Prompt: "Here are some translations from English to French: English: 'Good morning.' → French: 'Bonjour.' | English: 'How are you?' → French: 'Comment ça va?' Translate this sentence to French: 'What is your name?'"

Explanation: This is a few-shot prompt because you provided a couple of examples (translations) to show the model how to perform the task (translation). The model can now use these examples to understand how to translate the new sentence.

Expected Output: "Quel est ton nom?"

3. Chain-of-Thought Prompting

Definition: In chain-of-thought prompting, you ask the model to reason through a problem step by step. This approach helps the model break down complex tasks into smaller, logical steps, making the reasoning process more transparent.

A technique where the model is prompted to generate intermediate reasoning steps before arriving at a final answer, rather than jumping straight to a conclusion. This is especially effective for tasks requiring arithmetic, logic, or multi-step reasoning, since it mirrors how humans work through a problem, and reduces errors that come from skipping steps. It can be triggered explicitly ("Let's think step by step") or by including worked-out reasoning in few-shot examples.

Example:

Prompt: "Solve this math problem step by step: What is 12 multiplied by 9?"

Explanation: This is a chain-of-thought prompt because you are explicitly asking the model to go through the problem step by step. The model will not just give the final answer but will work through the process of solving it.

Expected Output:

- Step 1: "First, let's recall that 12 times 9 means adding 12, nine times."
- Step 2: "So, $12 + 12 + 12 + 12 + 12 + 12 + 12 + 12 + 12 = 108$."
- Final answer: "12 multiplied by 9 is 108."

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Summary of Differences

Technique	Description	Example
Zero-shot	The model does the task without examples. It's like giving it a question and expecting it to know the answer. Relies purely on pretrained knowledge	“What is the capital of France?”
Few-shot	You give a few examples to guide the model. This helps it understand the task better before asking for a new output.	“Translate ‘Good morning’ to French” (followed by examples)
Chain-of-thought	You ask the model to work through the task step by step, which is especially useful for solving problems logically. Forces step-by-step reasoning, not just examples	“Solve this math problem step by step.”

Prompt Design

Prompt Design refers to the process of creating and structuring the input given to a language model to achieve a desired output. It focuses on how you phrase and format your prompts to get the best results.

Prompt Design vs Prompt Engineering

1. Prompt Design Examples

Goal: Crafting clear, simple, and direct prompts to get the desired output from the model.

Example 1: Simple Question

Prompt Design: “What is the capital of France?”

Explanation: This is a simple and direct prompt. The model is asked for specific information — the capital of France. It's clear and leaves no room for ambiguity. The model will likely respond with “Paris.”

Example 2: Task-Oriented Instruction

Prompt Design: “Write a short paragraph about the benefits of exercise.”

Explanation: This prompt is clearly worded, asking the model to write a short paragraph. It's a straightforward instruction with no complex reasoning or multi-step thinking required.

2. Prompt Engineering Examples

Goal: Using more sophisticated techniques to refine the prompt and control the model's behavior. This includes things like few-shot, chain-of-thought, system instructions, and controlling output randomness.

Example 1: Few-Shot Prompting

Prompt Engineering (Few-shot): “Translate the following English sentences to French. English: ‘I am going to the store.’ French: ‘Je vais au magasin.’ English: ‘She is reading a book.’ French: ‘Elle lit un livre.’ English: ‘How are you today?’”

Explanation: In this case, you're providing examples (few-shot learning) to the model to help it understand how to translate sentences. By showing the pattern of English to French translation, the model is better equipped to translate the final sentence correctly. This technique improves the model's ability to follow a specific pattern or task.

Example 2: Chain-of-Thought Prompting

Prompt Engineering (Chain-of-Thought): "Solve this math problem step by step: What is 256 divided by 8?"

Explanation: In this example, you're asking the model to think through the problem step by step (chain-of-thought). Instead of just asking for the answer, you're guiding the model to break down the solution process. The model might respond with:

- Step 1: $256 \div 8 = 32$
- Final answer: 32

This approach ensures the model doesn't skip any important steps, which is especially useful for more complex tasks like math or reasoning problems.

Example 3: System Prompting (Instruction for Behavior)

Prompt Engineering (System Prompting): "You are an expert nutritionist. Your task is to give healthy eating advice based on someone's age and lifestyle. Please respond in a friendly, supportive tone."

Explanation: Here, you're setting the context for how the model should behave by using a system prompt. You're telling the model to act as an "expert nutritionist" and to maintain a "friendly, supportive tone" in its response. This controls the model's behavior, ensuring that it not only provides accurate information but also does so in a manner appropriate for the situation.

Example 4: Temperature Control (Adjusting Creativity)

Prompt: "Write a creative story about a time-traveling cat." | Temperature setting: 0.8 (Higher creativity)

Explanation: By setting the temperature to 0.8, you're telling the model to be more creative in its response. If you set the temperature lower (e.g., 0.2), the model's responses would be more predictable and formal. A higher temperature encourages a more varied and imaginative output, making it suitable for tasks like creative writing.

Comparison of Design vs Engineering Using the Same Task

Task: Generate a Business Idea for a Startup

Prompt Design (Simple):

"Give me a business idea for a tech startup."

Expected Output: "A software platform that helps businesses manage their projects more efficiently."

Explanation: This is a basic prompt asking for a business idea. It's clear, simple, and direct but might not yield highly innovative or specific results.

Prompt Engineering (Few-Shot + Chain of Thought):

"Here are some examples of successful tech startup ideas: 1. A platform that connects freelancers with businesses. 2. An app that helps users learn new skills via micro-courses. Based on these, give me a new business idea in the tech industry that focuses on improving remote team collaboration."

Expected Output: "A cloud-based tool that uses AI to automate team task management and enhance communication for remote teams."

Explanation: This approach is more sophisticated. You're providing examples (few-shot), narrowing down the context to tech startups (industry-specific), and using the chain-of-thought method to guide the model into thinking about the task in a particular way. This yields a more tailored and focused business idea.

Key Takeaways

- Prompt Design focuses on clarity and simplicity, ensuring the model understands exactly what you're asking for.
- Prompt Engineering is more advanced, involving optimization techniques, such as few-shot learning, chain-of-thought reasoning, and adjusting parameters like temperature to control output creativity.

In simple terms, Prompt Design is about creating clear and effective prompts, while Prompt Engineering is about optimizing those prompts to guide the model's behavior and improve results.

Image Prompting

- Image prompting is a form of prompt engineering used to guide an image generation model to produce a specific image output.
- An image prompt consists of three main parts — the image content, the art form and style, and the additional details — and generally follows this pattern:

```
[main subject of the image, description of action, state, mood],  
[art form, art style, artist references, if any],  
[additional settings, such as lighting, colors, framing].
```

Many permutations and combinations are available depending on the AI model being used. Some areas to think about:

- Art medium — Drawing, painting, ink, origami, mosaic, pottery, realistic and glazed
- Camera — Lens and perspective, camera settings
- Display and resolution — 8K, 4K, HD, 256×256, 512×512, 768×768
- Lighting — Types, display
- Material — Metal, cloth, glass, wood, liquids

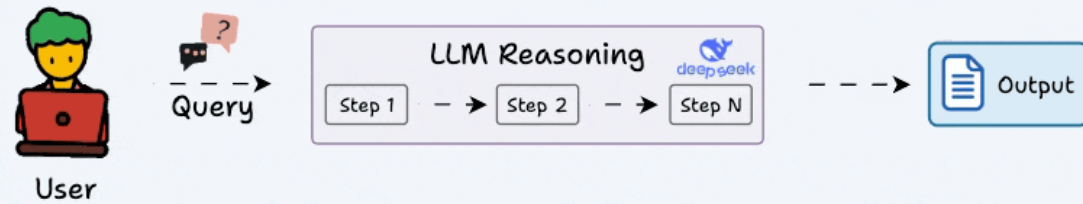
Example prompt:

```
Lion Roaring in a dense forest, angry mood, Other Animals in the distant  
background, realistic, colorful, 8K
```

3 prompting techniques for reasoning in LLMs

3 Prompting Techniques for Reasoning in LLMs

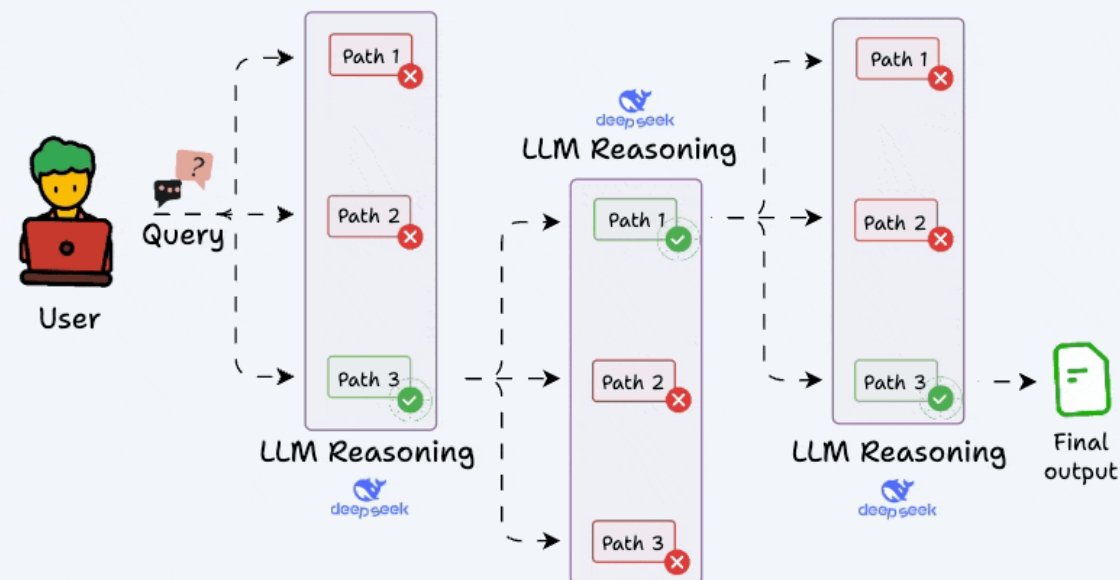
Chain of Thought (CoT)



Majority Voting over CoT



Tree of Thought



1) Chain of thought (CoT)

The simplest and most widely used technique.

Instead of asking the LLM to jump straight to the answer, we nudge it to reason step by step.



This often improves accuracy because the model can walk through its logic before committing to a final output.

For instance:

Q: If John has 3 apples and gives away 1, how many are left? Let's think step by step:

It's a simple example, but this tiny nudge can unlock reasoning capabilities that standard zero-shot prompting could miss.

For starters, zero-shot prompting means giving the model a task without any intermediate steps or examples. Ask a question and expect an answer.

Definition: Ask the model to reason **step by step** instead of jumping straight to the final answer. This breaks a complex problem into smaller, logical steps — and often improves accuracy on math/logic tasks.

Example prompt: "Solve this step by step: What is 12 multiplied by 9?"

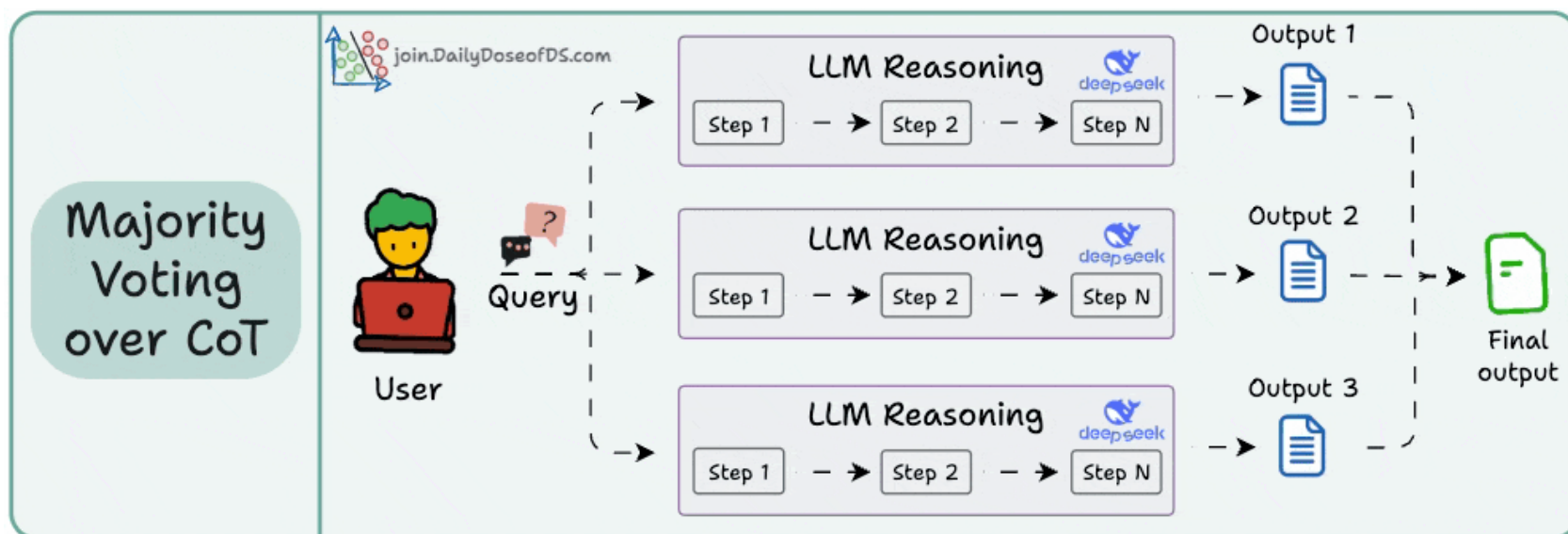
2) Self-consistency (or majority voting over CoT)

CoT is useful but not always consistent.

If you prompt the same question multiple times, you might get different answers depending on the temperature setting (we covered temperature in LLMs here).

Self-consistency embraces this variation.

You ask the LLM to generate multiple reasoning paths and then select the most common final answer.



It's a simple idea: when in doubt, ask the model several times and trust the majority.

This technique often leads to more robust results, especially on ambiguous or complex tasks.

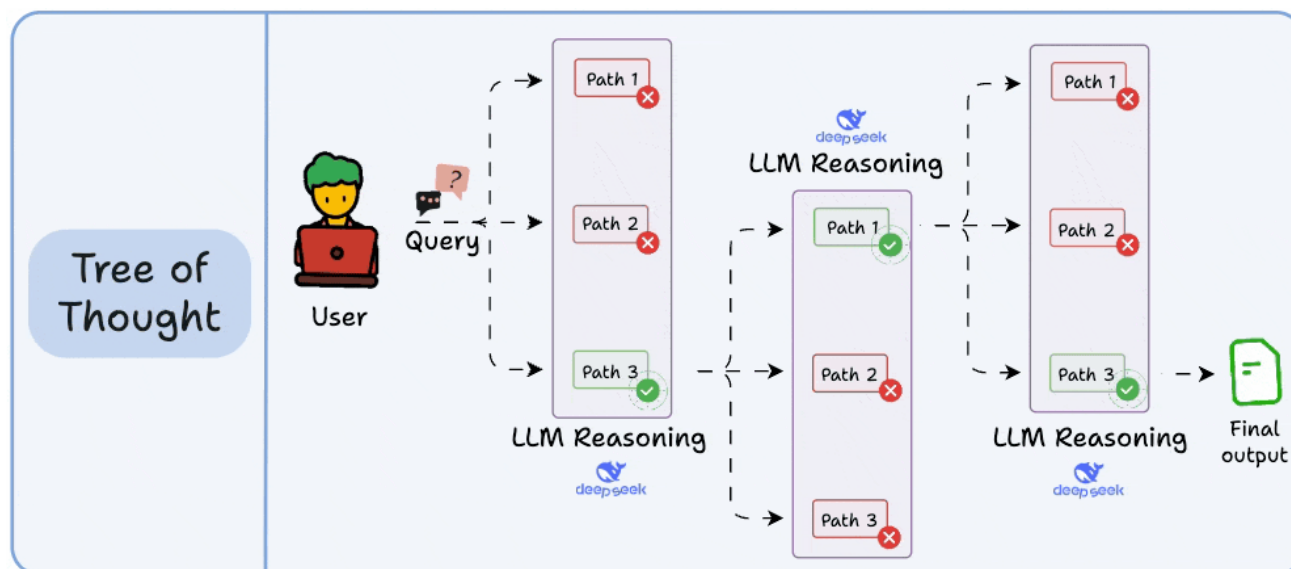
However, it doesn't evaluate how the reasoning was done—just whether the final answer is consistent across paths.

3) Tree of thoughts (ToT)

While Self-Consistency varies the final answer, Tree of Thoughts varies the steps of reasoning at each point and then picks the best path overall.

At every reasoning step, the model explores multiple possible directions. These branches form a tree, and a separate process evaluates which path seems the most promising at a particular timestamp.

Tree-of-Thought is a prompting technique where the model explores several possible reasoning paths at each step, like branches on a tree, instead of following just one line of thinking. It evaluates which branches look promising, drops the weak ones, and can backtrack — helping it solve problems that need planning or trial-and-error better than a single straight chain of reasoning would.



Think of it like a search algorithm over reasoning paths, where we try to find the most logical and coherent trail to the solution.

It's more compute-intensive, but in most cases, it significantly outperforms basic CoT.

Quick comparison:

Technique	Compute cost	Best for
Chain-of-Thought	Low	General multi-step reasoning
Self-Consistency	Medium-High (multiple samples)	Math/logic where errors are easy to catch by voting
Tree-of-Thought	High (search + evaluation)	Planning, puzzles, tasks needing exploration/backtracking

Want me to go deeper on any one of these — e.g., how to structure a ToT prompt, or how self-consistency voting is implemented in code?

Easy Way to Remember

Imagine you're trying to reach a destination.

Chain of Thought

You follow **one road**, carefully, step by step.

Home
|
Road
|
Destination



Self-Consistency

You ask **three friends** for directions.

If they all suggest the same route, you trust it.

Friend A ↙
Friend B + Same Route → Go
Friend C ↘



Tree of Thoughts

You open Google Maps.

It shows **multiple routes**, compares traffic, distance, and time, and helps you choose the best one.

Route A

Route B

Route C

↓

Compare

↓

Best Route



Summary

- **Chain of Thought (CoT):** Ask the model to reason **step by step**. It's simple, fast, and effective for math, coding, and logic.
- **Self-Consistency:** Generate **multiple independent reasoning paths** and choose the answer that appears most consistently. This improves reliability on difficult reasoning tasks.
- **Tree of Thoughts (ToT):** Explore **multiple solution branches**, evaluate each one, and select the best path. This is especially useful for planning, design, and strategy where there isn't a single obvious solution.

LangChain Basics: Chaining, Prompts, Messages & First Experiment

https://github.com/raffeemdai/RAG_BY_ME/blob/main/RAG-2-langchain-chaining-prompts-messages-notes.md

https://github.com/raffeemdai/LangChain_LangGraph/blob/main/imp-langchain_advanced_notes.md

1. Chaining — The Core Idea

The most basic way to talk to an LLM is:

prompt → llm → response

LangChain lets you **chain** these pieces together using the pipe operator `|` — same mental model as Unix pipes, where the output of one step feeds directly into the next.

chain = prompt | llm

Simple explanation: think of it like an assembly line. The prompt template fills in your question → passes it to the LLM → the LLM's answer comes out the other end.

Wanting structured output (JSON)

If you don't just want a plain-text answer but a structured one, you add another link to the chain:

```
chain = prompt | llm | json_parser
```

Simple explanation: the raw LLM response is just text. A parser sits at the end of the chain and converts that text into something usable in code — like a Python dictionary — instead of you having to manually clean up the string yourself.

1.1 Chaining — the core idea

The most basic way to talk to an LLM is:

```
prompt → llm → response
```

LangChain lets you **chain** these pieces together using the pipe operator `|` — the same mental model as Unix pipes, where the output of one step feeds straight into the next:

```
chain = prompt | llm
```

Simple picture: think of an assembly line. The prompt template fills in your question → passes it to the LLM → the LLM's answer comes out the other end.

If you don't want a plain-text answer but a structured one, you add another link:

```
chain = prompt | llm | json_parser
```

A raw LLM response is just text. A parser sits at the end of the chain and turns that text into something usable in code — like a Python dictionary — instead of you cleaning up the string by hand.

What is LCEL?

This `|` notation is called **LCEL — LangChain Expression Language**. It's LangChain's syntax for composing components (prompts, models, parsers, retrievers, tools) into a pipeline, where every piece speaks the same interface (`invoke`, `stream`, `batch`) so they can be linked together with `|`.

Simple explanation: LCEL is just a clean, readable way of saying "do this, then this, then this" — instead of writing lots of manual glue code, you snap components together like Lego blocks.

What is LCEL? This `|` notation is called **LCEL — LangChain Expression Language**. It's LangChain's syntax for composing components (prompts, models, parsers, retrievers, tools) into a pipeline, where every piece speaks the same interface (`invoke`, `stream`, `batch`) so they can be linked with `|`. It's a clean, readable way of saying "do this, then this, then this" — instead of writing manual glue code, you snap components together like Lego blocks.

2. Prompt — What You Send to the Model

A **prompt** is the input given to the LLM. Underneath, LLMs recognize different *types* of prompts, based on role:

Type	Purpose	Simple explanation
System Prompt	Defines the model's role/behavior	Like giving an employee their job description before they start work — "You are a customer support agent, only answer politely and from company docs."
User Prompt	The actual question being asked	This is you, the user, typing your actual question.

When you send this prompt to the model, you get a **response** back.

3. Messages — LangChain's Way of Organizing a Conversation

A real conversation isn't just one prompt and one response — it can go back and forth, involve tools, etc. LangChain generalizes every piece of a conversation into a **Message** object. There are **4 core types**:

LangChain Class	Represents	Simple explanation
SystemMessage	System Prompt	The "instructions" given once, before the conversation starts
HumanMessage	User Prompt	What the human typed
AIMessage	LLM's response	What the model replied with
ToolMessage	Result of a tool call	If the LLM asked to use a tool (like a calculator or search), this carries the tool's answer back to the LLM

Why bother with these classes instead of plain strings? Different LLM providers (OpenAI, Gemini, Anthropic) each expect conversation history in slightly different formats behind the scenes. LangChain's Message classes are a common, provider-agnostic way to represent a conversation — write it once, and LangChain translates it for whichever model you're using.

1.4 Quick recap (interview-ready)

- LCEL (`|`) chains components together — prompt → llm → parser — because they all implement the same Runnable interface (next section).
- Prompt = input to the LLM: a system part (role/behavior) + a user part (the actual ask).
- Messages are LangChain's typed, provider-agnostic way of representing a conversation: SystemMessage , HumanMessage , AIMessage , ToolMessage .
- Once basic invocation (llm.invoke([HumanMessage(...)])) works, LCEL (prompt | llm | parser) is the natural next step — reusable prompts, cleanly structured output.

Prompt Templates in LangChain

Theory, Concepts, and Practical Code Using the Euri API

Reference Notes

[PromptTemplate vs ChatPromptTemplate — Thakur Rana \(Medium\)](#)

https://github.com/raffeemdai/LangChain_LangGraph/blob/main/prompt_engineering_notes_euri.md **section 7**

https://github.com/raffeemdai/LangChain_LangGraph/blob/main/LangChain_Syntax_Theory_Guide.md

1. What Is a Prompt Template?

A Prompt Template in LangChain is a reusable prompt with {placeholders} — you define the fixed structure once, then fill in dynamic values at runtime. Instead of hardcoding strings, this gives you reusability, validation, and composability into chains.

Rather than scattering raw f-strings across a codebase, a Prompt Template formalizes a prompt into an object with declared inputs, built-in validation, and a clean separation between prompt authoring and application logic.

1.1 The Two Core Template Types

Type	What It Produces	When To Use
PromptTemplate	One formatted string	Simple, single-turn prompts
ChatPromptTemplate	List of role-tagged messages	Chat models needing persona / system behavior

The pipe operator template | llm (LangChain Expression Language, or LCEL) is what wires the filled prompt straight into the model. This works with any OpenAI-compatible LLM — including free tiers like Euri — because LangChain abstracts the model provider behind a common Runnable interface.

2. Step 1 — Euri Setup (One-Time)

Euri is a free, OpenAI-compatible API endpoint. Because it follows the same request/response contract as OpenAI, it plugs directly into LangChain's ChatOpenAI class without any custom integration code.

2.1 Environment Variables (.env)

```
EURI_API_KEY=your_key_here
EURI_BASE_URL=https://api.euron.one/api/v1/euri
EURI_MODEL=gemini-2.5-flash
```

2.2 Reusable Client (euri_client.py)

```
# euri_client.py
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

llm = ChatOpenAI(
    model=os.getenv("EURI_MODEL", "gemini-2.5-flash"),
    api_key=os.getenv("EURI_API_KEY"),
    base_url=os.getenv("EURI_BASE_URL"),
)
```

Note: Every example below simply imports llm from this file, keeping model configuration in one place.

3. Step 2 — PromptTemplate (Simple String)

PromptTemplate is used for plain, single-turn prompts. It fills {placeholders} using standard Python format syntax and produces one formatted string.

```
from langchain_core.prompts import PromptTemplate
from euri_client import llm

template = PromptTemplate.from_template(
    "Explain {topic} to a {audience} in 2 sentences."
)

chain = template | llm
result = chain.invoke({"topic": "vector embeddings", "audience": "5-year-old"})
print(result.content)
```

Here, template | llm is LCEL: the template fills in the variables first, then pipes the finished text straight into the model.

4. Step 3 — ChatPromptTemplate (System + Human Roles)

ChatPromptTemplate is designed for chat-based models that expect a sequence of role-tagged messages — system, human, and ai — rather than a single string. **This is the standard for modern chat models such as GPT-4, Claude, and Gemini.**

```
from langchain_core.prompts import ChatPromptTemplate
from euri_client import llm

chat_template = ChatPromptTemplate.from_messages([
    ("system", "You are a concise programming tutor who always gives a code example."),
    ("human", "Explain {concept} in Python."),
])

chain = chat_template | llm
result = chain.invoke({"concept": "decorators"})
print(result.content)
```

The system message sets persona and behavior once; the human message carries the dynamic ask. This mirrors how real chat APIs structure conversations.

5. Step 4 — Reusable Prompt Library Pattern

In production applications, templates are kept in a dedicated module and imported wherever needed. This keeps prompts version-controlled, testable, and swappable without touching business logic.

```
# prompts.py
from langchain_core.prompts import ChatPromptTemplate

SUMMARIZER_PROMPT = ChatPromptTemplate.from_messages([
    ("system", "Summarize input text in exactly 3 bullet points."),
```



```
    ("human", "{text}"),
])

# main.py
from prompts import SUMMARIZER_PROMPT
from euri_client import llm

chain = SUMMARIZER_PROMPT | llm
print(chain.invoke({"text": "LangChain is a framework for building LLM apps."}).content)
```

6. Summary

- PromptTemplate → single formatted string, best for simple single-turn prompts.
- ChatPromptTemplate → list of role-tagged messages, best for chat models needing persona or system behavior.
- LCEL's pipe operator (template | llm) connects any prompt template to any OpenAI-compatible LLM, including Euri.
- Keeping templates in a dedicated prompts.py module is the recommended pattern for production LangChain applications.

Interview one-liner: *A LangChain prompt template is a parameterized prompt — PromptTemplate for plain strings, ChatPromptTemplate for role-based chat messages. You fill placeholders at runtime and pipe the result into any LLM via LCEL (template | llm) — the LLM itself is swappable, whether it's OpenAI, Anthropic, or a free OpenAI-compatible endpoint like Euri, because LangChain abstracts the provider behind a common interface.*

7. Reference

Reference: [PromptTemplate vs ChatPromptTemplate: Understanding and Invoking Them in LangChain — Thakur Rana, Medium](#)

LangChain Runnable

Simple Notes — Interview Focused

PART 1: THE BASICS

What is a Runnable?

A Runnable is just a rule LangChain follows:

"If something is a Runnable, it can always be run using the same 3 commands — invoke(), batch(), stream() — no matter what it actually does inside."

That's it. That's the whole idea.

Examples of things that are Runnables:

- A prompt template

- A chat model (OpenAI, Gemini, Euri, etc.)
- An output parser
- A retriever
- A custom Python function (once wrapped)
- An entire chain (prompt | llm | parser is also one big Runnable)
-

Analogy: It's like a USB port. A mouse, a keyboard, and a USB drive are all completely different devices — but they all plug into the same USB port and get recognized the same way. A Runnable is LangChain's "USB port" — one common way to plug anything in.

The 3 Methods You Actually Need to Know

Method	Meaning	Everyday example
invoke()	Run once, get one answer	Ask one question, get one answer
batch()	Run many inputs at once	Ask a list of questions, get a list of answers
stream()	Get the answer piece by piece	Watch a reply being typed live, instead of waiting

(There's also `ainvoke()`, the *async* version of `invoke()` — same idea, just non-blocking.)

Why this matters: because a prompt, a model, and a parser all support these same methods, you can chain them together with `|`:

```
chain = prompt | llm | parser
chain.invoke({"topic": "AI"})
```

Analogy: Think of Lego bricks. Every brick has the same connector shape, so any brick snaps onto any other brick. **Runnables all share the same "connector" (invoke/batch/stream), so any Runnable can snap onto any other Runnable using `|`.**

Quick Way to Check: "Is This a Runnable?"

Yes, it's a Runnable ✓	No, it's not ✗
Prompt Template	A Document object (just data)
Chat Model	A TextSplitter (one-time setup tool)
Output Parser	A DocumentLoader (one-time setup tool)
Retriever (vectorstore.as_retriever())	VectorStore itself (before calling .as_retriever())
A whole chain (prompt llm parser)	The final answer/response you get back

Simple rule to remember it:

- If it **does work** at the moment you ask a question → it's a Runnable.
- If it's **raw data**, or a **one-time setup step** (loading files, splitting text, building a database) → it's not.

2. Runnable — The Interface That Makes LCEL Possible

Runnable is the base class almost everything in LangChain inherits from. Anything that is a Runnable can be executed the exact same way, no matter what it actually does internally.

The 4 core methods every Runnable shares

Method	Purpose
invoke()	Run once, synchronously (single input → single output)
ainvoke()	Async version of invoke()
batch()	Run multiple inputs in parallel
stream()	Yield the output incrementally as it's generated

Why this matters: once you know these 4 methods, you can run *anything* in LangChain — a model, a prompt, a parser, a retriever, or an entire multi-step chain — the same way.

•

PART 2: INTERVIEW QUESTIONS & MODEL ANSWERS

Q1. What is a Runnable in LangChain?

Answer: A Runnable is the base interface almost everything in LangChain implements. It guarantees that any component — a prompt, a model, a parser, a retriever, or a full chain — can be run the same way, using `invoke()`, `batch()`, and `stream()`. This shared interface is what allows different components to be connected together predictably.

Q2. What is LCEL, and how does the `|` operator actually work?

Answer: LCEL (LangChain Expression Language) is the syntax for composing Runnables using the pipe operator `|`. It works because every component on either side of `|` implements the Runnable interface — so LangChain knows exactly how to take the output of one step and feed it as the input to the next. Since the resulting chain is itself a Runnable, you can call `.invoke()`, `.batch()`, or `.stream()` on the entire pipeline, not just on individual pieces.

Q3. Is a VectorStore a Runnable? Why or why not?

Answer: No. A raw VectorStore (like Chroma or FAISS) is not a Runnable — it's just a database of vectors. You have to call `.as_retriever()` on it first, which wraps it into a Retriever object. The Retriever is a Runnable, because it exposes `.invoke(query)` and returns relevant documents.

Q4. If you swap ChatOpenAI for ChatAnthropic (or a different provider), why doesn't the rest of your code need to change?

Answer: Because all chat model classes inherit from the same base Runnable interface (through `BaseChatModel`). Regardless of the actual provider behind the scenes, `.invoke()`, `.batch()`, and `.stream()` behave identically. Only the model-creation line (API key, base URL, model name) changes — the prompt, chain, and parsing logic stay exactly the same.

Q5. How do you use a plain Python function inside a LangChain chain?

Answer: Wrap it with `RunnableLambda`. A normal Python function doesn't implement the Runnable interface on its own, so it can't be piped with `|`. `RunnableLambda(my_function)` wraps it so it behaves like any other Runnable and can be chained normally.

```
from langchain_core.runnables import RunnableLambda
uppercase = RunnableLambda(lambda x: x.upper())
```

Q6. How do you run two chains at the same time and combine their results?

Answer: Use `RunnableParallel`. It runs multiple Runnables concurrently against the same input and returns a dictionary with each result labeled by key.

```
from langchain_core.runnables import RunnableParallel
parallel = RunnableParallel({"summary": summary_chain, "quiz": quiz_chain})
```

Q7. In a RAG chain, how does the original user question reach the final prompt alongside the retrieved documents?

Answer: Using `RunnablePassthrough()`. It simply forwards the input unchanged. In a RAG setup, you typically run the retriever on the question to get context, while `RunnablePassthrough()` carries the original question through untouched — so both the context and the original question are available together for the final prompt.

```
{"context": retriever | format_docs, "question": RunnablePassthrough()}
```

Q8. What's the difference between a Runnable and a non-Runnable component, in one sentence?

Answer: A Runnable does work at query time (takes an input, produces an output, right when you ask something); a non-Runnable is either raw data (like a Document) or a one-time setup tool used before the chain runs (like a TextSplitter or DocumentLoader).

PART 3: ONE-LINE MEMORY HOOKS (For Quick Revision)

Term	One-liner
Runnable	Same interface (invoke/batch/stream) for everything
invoke()	Run once
batch()	Run a list at once
stream()	Get the answer piece by piece
LCEL ()	Snaps Runnables together, like Lego
RunnableLambda	Wraps a plain function so it can join a chain
RunnableParallel	Runs multiple chains at the same time
RunnablePassthrough	Forwards input untouched
Not a Runnable	Raw data / one-time setup (Document, Loader, Splitter, VectorStore)

If you remember only one thing: Runnable = the shared interface that lets LangChain snap different pieces together with |, and run any of them the same way.